```
import numpy as np
import random

# Distance matrix representing amino acid positions
distance = np.array([
    [0, 2, 9, 10, 7],
    [2, 0, 6, 4, 3],
    [9, 6, 0, 8, 5],
    [10, 4, 8, 0, 6],
    [7, 3, 5, 6, 0]
])

# Parameters
num_ants = 5
num_iterations = 50
alpha = 1
beta = 2
evaporation = 0.5
Q = 100

n = len(distance)

# Initialize pheromone
pheromone = np.ones((n, n))

best_path = None
best_distance = float('inf')

# Ant Colony Optimization
for iteration in range(num_iterations):

    all_paths = []
    all_distances = []

    for ant in range(num_ants):

        path = [random.randint(0, n-1)]

        while len(path) < n:
```

```
            current = path[-1]
            unvisited = [i for i in range(n) if i not in path]

            probabilities = []

            for city in unvisited:
                p = (pheromone[current][city] ** alpha) * \
                    ((1 / distance[current][city]) ** beta)
                probabilities.append(p)

            total = sum(probabilities)
            probabilities = [p / total for p in probabilities]

            next_city = random.choices(
                unvisited, weights=probabilities
            )[0]

            path.append(next_city)

        # Calculate total tour distance
        total_distance = sum(
            distance[path[i]][path[i+1]]
            for i in range(n-1)
        )

        total_distance += distance[path[-1]][path[0]]

        all_paths.append(path)
        all_distances.append(total_distance)

        if total_distance < best_distance:
            best_distance = total_distance
            best_path = path.copy()

    # Evaporate pheromones
    pheromone *= (1 - evaporation)

    # Update pheromones
    for path, d in zip(all_paths, all_distances):
        for i in range(n):
```

```
            a = path[i]
            b = path[(i+1) % n]
            pheromone[a][b] += Q / d
            pheromone[b][a] += Q / d

# Display result
print("Best Protein Residue Path:")
print([i + 1 for i in best_path])

print("Minimum Distance:", best_distance)
```

OUTPUT:

```
Best Protein Residue Path:
[2, 1, 5, 3, 4]
Minimum Distance: 26
```